



Conditional Statements

(if, else if, else, switch)

1. What are Conditional Statements?

Conditional statements in JavaScript allow your program to **make decisions** based on conditions.

They help execute different blocks of code depending on whether a condition is true or false.

2. Types of Conditional Statements in JavaScript

Sr	Statement	Use
1	If	Run a block of code only if a condition is true
2	else	Run an alternate block if the condition is false
3	else if	Test multiple conditions
4	(Also: switch)	Alternative to <code>else if</code> for specific values

Syntax :

1. if Statement :

- condition: A boolean expression (true/false).
- If the condition is **true**, the code inside { } runs.
- If it's **false**, the code is skipped.

```
let age = 20;

if (age >= 18) {
  console.log("You are eligible to vote.");
}
```

Checking if a user is adult or minor. ↑

2. if...else Statement

The if...else statement allows you to execute **one block of code** if a **condition is true**, and **another block** if the **condition is false**.

```
let isLoggedIn = false;

if (isLoggedIn) {
  console.log("Welcome back!");
} else {
  console.log("Please log in first.");
}
```

Show different UIs for logged-in vs guest users. ↑

3. if...else if...else Statement

```
let marks = 75;

if (marks >= 90) {
  console.log("Grade: A");
} else if (marks >= 75) {
  console.log("Grade: B");
} else {
  console.log("Grade: C");
}
```

Assign grades based on multiple conditions. ↑

Examples :

Sr	Use	Example
1	Age restriction	if (age >= 18)
2	Login check	if (isLoggedIn)
3	Form validation	if (!inputValue)
4	Pricing logic	if (price > 1000) { applyDiscount(); }
5	Role-based access	if (role === 'admin')
6	Responsive design condition	if (window.innerWidth < 768)

Nested Conditions

```
let user = {
  isLoggedIn: true,
  isAdmin: false,
};

if (user.isLoggedIn) {
  if (user.isAdmin) {
    console.log("Welcome Admin");
  } else {
    console.log("Welcome User");
  }
}
```

Handle multi-level roles. ↑

Common Mistakes to Avoid

Sr	Mistake	Better Way
1	Using = instead of == or ===	Always use === for strict comparison
2	Forgetting curly braces { }	Use braces for clarity and bug prevention
3	Complex nested if conditions	Break into functions or use switch for clarity

Practices

- Use === instead of ==
- Don't over-nest if blocks — break logic into small functions
- Prefer else if over multiple ifs
- For many exact value matches, consider using switch

Summary :

Sr	Condition	Result
1	if (true)	Executes block
2	if (false)	Skips block
3	if...else	Executes one of two blocks
4	else if chain	Checks multiple options in order

Use :

E-commerce Discount System :

```
let price = 1200;
let isMember = true;

if (price > 1000 && isMember) {
  console.log("Apply 20% member discount");
} else if (price > 1000) {
  console.log("Apply 10% discount");
} else {
  console.log("No discount");
}
```

Member gets more benefits than non-member. ↑

Dark Mode Based on Time :

```
let currentHour = new Date().getHours();

if (currentHour >= 18 || currentHour <= 6) {
  console.log("Enable Dark Mode");
} else {
  console.log("Enable Light Mode");
}
```

UI changes based on system time. ↑

Mobile App Logic – Location Access

```
let isGPSOn = true;
let isPermissionGranted = false;

if (!isGPSOn) {
  console.log("Please enable GPS");
} else if (!isPermissionGranted) {
  console.log("Allow location permission");
} else {
  console.log("Accessing location...");
}
```

Step-by-step user permission checks. ↑

System Check

```
1 let paymentMode = 'upi';
2
3 if (paymentMode === 'card') {
4   console.log('Processing card payment...');
5 } else if (paymentMode === 'upi') {
6   console.log('Processing UPI payment...');
7 } else if (paymentMode === 'cod') {
8   console.log('Order confirmed. Pay on delivery.');
```

else if chains are better than nested if. ↑

Real Interview Pattern Example

```
1 let experience = 2;
2 let skills = ['JavaScript', 'React'];
3
4 if (experience >= 3 && skills.includes('Node.js')) {
5   console.log('Assign Full Stack Project');
```

Matches HR interview logic in real companies. ↑

Bonus: if Without Braces (Avoid!)

```
if (isLoggedIn)
  console.log("Welcome");
// BAD: Can lead to confusion in larger apps
```

Always prefer: ↓

```
if (isLoggedIn) {
  console.log("Welcome");
}
```

if as Expression in Modern JS (Ternary Alternative)

```
const message = isLoggedIn ? "Welcome!" : "Please log in.";
console.log(message);
```

Use for short conditions; otherwise prefer full `if...else`. ↑

Multiple Conditions with Logical Operators

Combining Conditions Using `&&` and `||`

```
let age = 25;
let hasLicense = true;

if (age >= 18 && hasLicense) {
  console.log("You can drive a car");
} else {
  console.log("Not eligible");
}
```


Government form validations / Driving eligibility ↑

else if Chain with Value Ranges

```
let temperature = 42;

if (temperature > 40) {
  console.log("Heatwave Alert!");
} else if (temperature > 30) {
  console.log("Hot Day");
} else if (temperature > 20) {
  console.log("Pleasant Weather");
} else {
  console.log("Cold Weather");
}
```

Weather apps, health apps, climate monitoring. ↑

if inside Loops (Dynamic Decisions)

```
let scores = [95, 67, 45, 88];

for (let score of scores) {
  if (score >= 90) {
    console.log(`${score} - Grade A`);
  } else if (score >= 75) {
    console.log(`${score} - Grade B`);
  } else {
    console.log(`${score} - Needs Improvement`);
  }
}
```

Marksheet generation / leaderboard systems ↑

Using if with Functions

```
function login(user) {  
  if (!user) {  
    return "User not found";  
  }  
  if (!user.isActive) {  
    return "Account is deactivated";  
  }  
  return "Welcome back!";  
}
```

Authentication, APIs, backend logic ↑

Conditional Execution Without else (Early Return Pattern)

```
function processPayment(amount) {  
  if (amount <= 0) {  
    console.log("Invalid amount");  
    return;  
  }  
  
  console.log("Processing payment...");  
}
```

Use for cleaner code with fewer nested blocks — preferred in production-level code. ↑

Comparing Truthy/Falsy Without ===

```
let value = "";

if (value) {
  console.log("Has value");
} else {
  console.log("Empty or falsy");
}
```

Falsy values in JS:

- false
- 0
- "" (empty string)
- null
- undefined
- NaN

Nested if...else if vs switch (When to use what?)

- Use if...else if when you're **checking ranges or conditions**
- Use switch when you're checking **specific fixed values**

```
let role = "admin";

switch (role) {
  case "admin":
    console.log("Admin Panel Access");
    break;
  case "editor":
    console.log("Editor Dashboard");
    break;
  default:
    console.log("Guest Access");
}
```

Mistakes

Sr	Mistake	Correct Way
1	Using = instead of ==	Use === for strict comparison
2	Nested ifs too deeply	Use early return or switch
3	Checking falsy loosely	Know the truthy/falsy rules
4	Forgetting else if fallback	Always include a final else if needed

Conditional Statements with Functions + Return Values

```
function getDiscount(userType) {  
  if (userType === "premium") return "20% discount";  
  else if (userType === "member") return "10% discount";  
  else return "No discount";  
}  
  
console.log(getDiscount("guest"));
```

Pricing engines, checkout flows, subscription logic ↑

if inside Array.map() / Array.forEach()

```
1 let users = [  
2   { name: 'Nadeem', online: true },  
3   { name: 'ABC', online: false },  
4 ];  
5  
6 users.forEach(user => {  
7   if (user.online) {  
8     console.log(`${user.name} is online`);  
9   } else {  
10    console.log(`${user.name} is offline`);  
11  }  
12 });  
13
```

Status displays in UI, dashboards, user lists ↑

if inside API Response Handling

```
fetch("/api/data")
  .then(res => res.json())
  .then(data => {
    if (data.status === "success") {
      console.log("Data loaded");
    } else if (data.status === "error") {
      console.error("Server error");
    } else {
      console.warn("Unknown response");
    }
  });
```

Real-world frontend apps (React, Vue, etc.) ↑

if + Ternary Chain Mix

```
let role = "editor";

let access =
  role === "admin" ? "Full Access" :
  role === "editor" ? "Edit Access" :
  role === "viewer" ? "Read-only" :
  "No Access";

console.log(access);
```

readability matters! ↑

Conditional State Update

```
function updateScore(score) {  
  if (score > 90) {  
    setBadge("Gold");  
  } else if (score > 70) {  
    setBadge("Silver");  
  } else {  
    setBadge("Bronze");  
  }  
}
```

React apps, dynamic UI updates ↑

Dynamic Content Rendering Based on Conditions

```
function renderPage(user) {  
  if (!user) return "<p>Please log in</p>";  
  else if (user.isAdmin) return "<h1>Welcome Admin</h1>";  
  else return `<h1>Welcome ${user.name}</h1>`;  
}
```

Server-side rendering (SSR), templates ↑

Use of if for Feature Flags

```
const isFeatureEnabled = true;  
  
if (isFeatureEnabled) {  
  enableNewNavbar();  
}
```

Feature toggles, A/B testing ↑

Reverse Thinking with if

```
if (!user.isActive) {  
  console.log("Account suspended");  
  return;  
}  
// Continue with normal logic...
```

Early return = Clean code ↑

if with Permission Checks

```
if (user.isLoggedIn && user.role === "admin") {  
  console.log("Access granted");  
} else {  
  console.log("Access denied");  
}
```

Role-based access control ↑

Browser Detection Example

```
let isSafari = /^(?!chrome|android).)*safari/i.test(navigator.userAgent);

if (isSafari) {
  console.log("Apply Safari-specific fix");
}
```

Cross-browser compatibility checks ↑

UI Interaction – Button Enable/Disable

```
let isValid = false;

if (isValid) {
  document.querySelector("#submit").disabled = false;
} else {
  document.querySelector("#submit").disabled = true;
}
```

Form handling, UX behavior ↑

Summary

Sr	Statement Type	Syntax	Description	Use
1	If	if (age >= 18) { ... }	Executes block only if condition is true	Checking age eligibility
2	if...else	if (loggedIn) {...} else {...}	Executes one of two blocks based on condition	Logged in vs Guest UI
3	if...else if...else	if (score>90)... else if (...)... else {...}	Handles multiple conditions in sequence	Grading system, discounts
4	Nested if	if (...) { if (...) { ... } }	if inside another if block	Role + Status check
5	if with && operator	if (isAdmin && isActive)	Both conditions must be true	Admin access check
6	if with ! operator	if (!isLoggedIn)	Executes if condition is false	Access restriction
7	Early return with if	if (!valid) return;	Stops function execution early	Form validation, cleaner code
8	if inside loop	array.forEach(el => { if (...) {...} })	Apply logic on each item	Score evaluation, list filter
9	if inside function	function checkUser(u) { if (...) {...} }	Conditional logic within reusable blocks	Reusability & clean flow
10	Ternary alternative	let msg = isValid ? 'Yes' : 'No';	Short version of if...else	Inline conditions
11	if for UI rendering	if (user) return <Dashboard />;	Show content based on user	React/Frontend JSX logic
12	if for error handling	if (!response.ok) { throw Error(...) }	Handle API errors before proceeding	Fetch/axios responses
13	if for feature flags	if (featureXEnabled) { ... }	Toggle features dynamically	A/B testing, beta toggles
14	if with switch suggestion	Use switch() for fixed values	Cleaner than multiple else ifs	Role-based UI

Mistake to Avoid :

Sr	❌ Common Mistake	Fix
1	Using = instead of ==	Always use === for comparison
2	Over-nesting if blocks	Use early return or switch for clarity
3	Forgetting final else	Always provide fallback in decision chains
4	Ignoring falsy values	Know JS falsy rules: false, 0, "", null, undefined, NaN

Enjoyed this guide? Support our team.

If you found value in this guide, help it reach others who need it.

A simple **like** 👍 , **comment** or **repost** makes a big difference — and keeps creators like us motivated to share more. ❤️

Let's keep learning. Together. 😊❤️

Interested in JavaScript mentorship?

If you'd like to learn JavaScript with **live sessions** on **Google Meet** or **Zoom** — feel free to reach out!

You can also email us at ✉️ reactnadeem@gmail.com

Let's learn, build, and grow together. 🚀💻